



State management

State management helpt ons de
status van de applicatie bij te
houden.

State management



De huidige
status waarin de
applicatie en zijn
data zich
bevindt

Beheren van
(user) acties die
de status
beïnvloeden

**De status van de applicatie
is een heel breed gegeven**

- Data van een component
- Loading status van een API call
- Error status van een API call
- Ingelogde user
- ...

De twee basisprincipes van state management

Immutability

Unidirectional data flow

Immutability slaat op **hoe de data**
bewaard en hoe er met de **data**
geïnterageerd wordt.

Mutable of immutable?

Een “**mutable**” is een variable die aangepast kan worden. In JavaScript zijn enkel **objects** en **arrays** mutable, **primitives** (*string, boolean, number, ...*) zijn dat per definitie niet.

Primitive (string)

```
const test1 = 'Hello world';  
  
console.log(test1); // Result: Hello world  
  
test1 = 'Hello EHB'; // Result:
```

```
const test1 = 'Hello world';
```

```
console.log(test1); // Result: Hello world
```

```
test1 = 'Hello EHB'; // Result: Error: Assignment to constant variable.
```

```
let test2 = 'Hello world';  
  
console.log(test2); // Result: Hello world  
  
test2 = 'Hello EHB';  
  
console.log(test2); // Result:
```

```
let test2 = 'Hello world';  
  
console.log(test2); // Result: Hello world  
  
test2 = 'Hello EHB';  
  
console.log(test2); // Result: Hello EHB
```

Array

```
const test1 = ['Hello', 'World'];  
console.log('test1', test1); // Result: ["Hello", "World"]
```

```
const test2 = test1;  
test2.push('EHB')  
console.log('test1', test1); // Result:  
console.log('test2', test2); // Result:
```

```
const test1 = ['Hello', 'World'];  
console.log('test1', test1); // Result: ["Hello", "World"]  
  
const test2 = test1;  
test2.push('EHB')  
console.log('test1', test1); // Result: ["Hello", "World", "EHB"]  
console.log('test2', test2); // Result: ["Hello", "World", "EHB"]
```


Object

```
const developer1 = {  
  name: 'Iben',  
  lastName: 'Van de Veire',  
};  
  
console.log('developer1:', developer1.name); // Result: "developer1 Iben"  
  
const developer2 = developer1;  
  
developer2.name = 'Jasper';  
  
console.log('developer1:', developer1.name); // Result:  
console.log('developer2:', developer2.name); // Result:
```

```
const developer1 = {  
  name: 'Iben',  
  lastName: 'Van de Veire',  
};  
  
console.log('developer1:', developer1.name); // Result: "developer1 Iben"  
  
const developer2 = developer1;  
  
developer2.name = 'Jasper';  
  
console.log('developer1:', developer1.name); // Result: "developer1 Jasper"  
console.log('developer2:', developer2.name); // Result: "developer2 Jasper"
```

Wanneer iets **mutable** is, past de copy het origineel ook aan.

Wanneer iets **immutable** is, wijzigt de copy het origineel niet.

Hoe op te lossen?

“Een kopie maken”:

Arrays:

- Spread operator (...)
- `Array.concat()`
- `Array.map()`
- `Array.filter()`
- ...

Objects:

- Spread operator (...)
- `Object.assign()`

Array

```
const test1 = ['Hello', 'World'];  
console.log('test1', test1); // Result: ["Hello", "World"]  
const test2 = ['EHB'];  
console.log('test2', test2); // Result: ["EHB"]  
  
const test3 = test1.concat(test2);  
console.log('test1', test1); // Result:  
console.log('test2', test2); // Result:  
console.log('test3', test3); // Result:  
  
const test4 = [...test1, ...test2];  
console.log('test4', test4) // Result:
```

```
const test1 = ['Hello', 'World'];  
console.log('test1', test1); // Result: ["Hello", "World"]  
const test2 = ['EHB'];  
console.log('test2', test2); // Result: ["EHB"]  
  
const test3 = test1.concat(test2);  
console.log('test1', test1); // Result: ["Hello", "World"]  
console.log('test2', test2); // Result: ["EHB"]  
console.log('test3', test3); // Result: ["Hello", "World", "EHB"]  
  
const test4 = [...test1, ...test2];  
console.log('test4', test4) // Result: ["Hello", "World", "EHB"]
```


Object

```
const developer1 = {  
  name: 'Iben',  
  lastName: 'Van de Veire',  
};  
  
console.log('developer1:', developer1.name); // Result: "developer1 Iben"  
  
const developer2 = Object.assign({}, developer1);  
  
developer2.name = 'Jasper';  
  
console.log('developer1:', developer1.name); // Result:  
console.log('developer2:', developer2.name); // Result:  
  
const developer3 = { ...developer1 };  
  
developer3.name = 'Jasper';  
  
console.log('developer1:', developer1.name); // Result:  
console.log('developer3:', developer3.name); // Result:
```

```
const developer1 = {  
  name: 'Iben',  
  lastName: 'Van de Veire',  
};  
  
console.log('developer1:', developer1.name); // Result: "developer1 Iben"  
  
const developer2 = Object.assign({}, developer1);  
  
developer2.name = 'Jasper';  
  
console.log('developer1:', developer1.name); // Result: "developer1 Iben"  
console.log('developer2:', developer2.name); // Result: "developer2 Jasper"  
  
const developer3 = { ...developer1 };  
  
developer3.name = 'Jasper';  
  
console.log('developer1:', developer1.name); // Result: "developer1 Iben"  
console.log('developer3:', developer3.name); // Result: "developer2 Jasper"
```

Immutability betekent dus dat we
nooit direct de data aanpassen.

De voordelen van immutability

- Voorspelbaar
- Testbaar
- Side-effects vermijden

Helpers

- <https://immutable-js.com/>

Unidirectional data flow omvat hoe
data zich gedraagt tussen
components

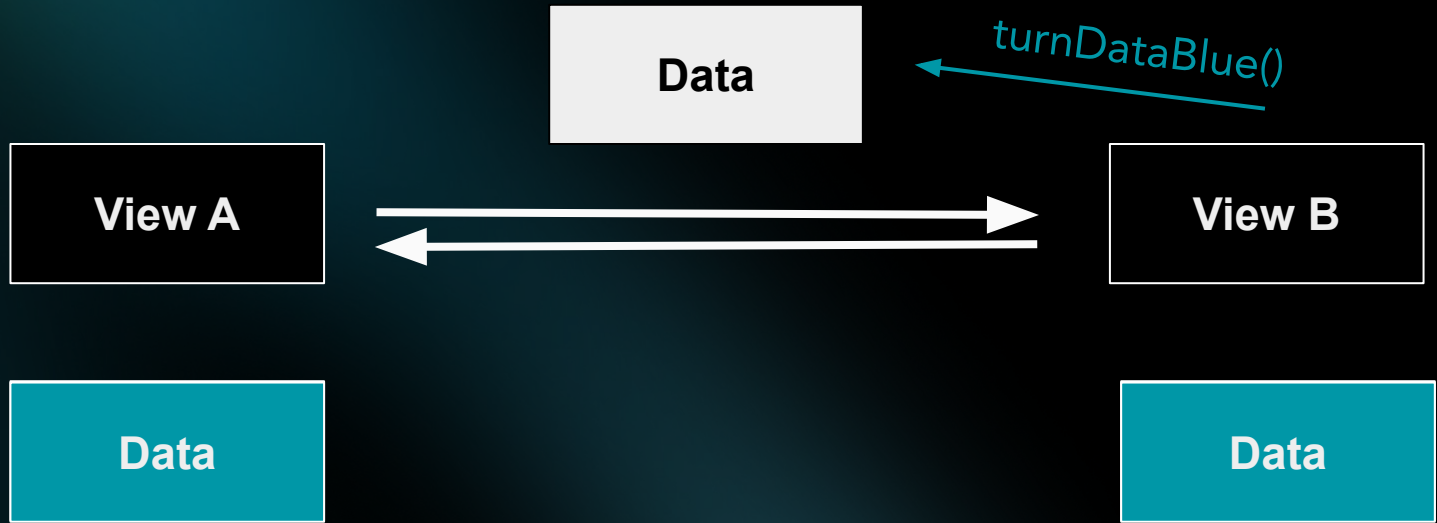
We maken een onderscheid tussen **bidirectional** en **unidirectional data flow**

Concreet kijken we hoe data zich van parent View A naar child View B verplaatst.

Wanneer een wijziging aan de data in View B ook een wijziging veroorzaakt in View A, dan spreken we over **bidirectional data flow** of **two way binding**.

Wanneer een wijziging aan de data in View B geen wijziging veroorzaakt in View A, dan spreken we over **unidirectional data flow** of **one way binding**.

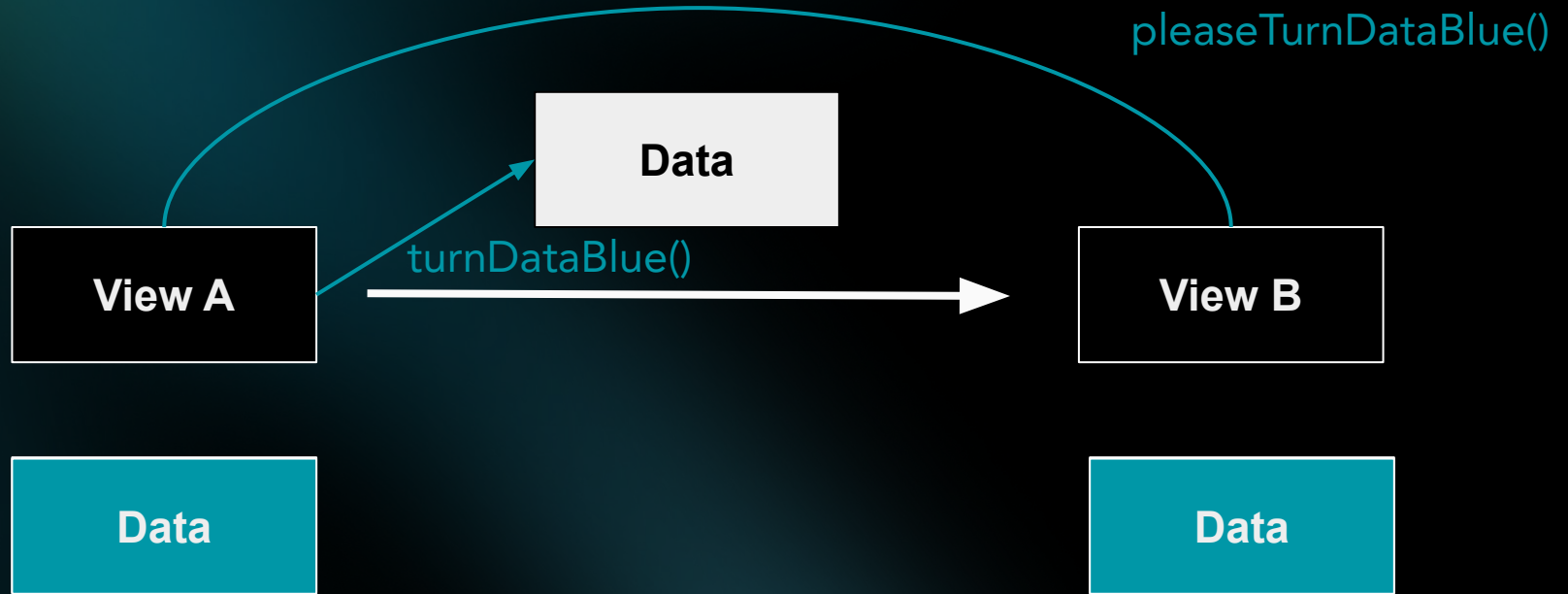
Bidirectional data flow



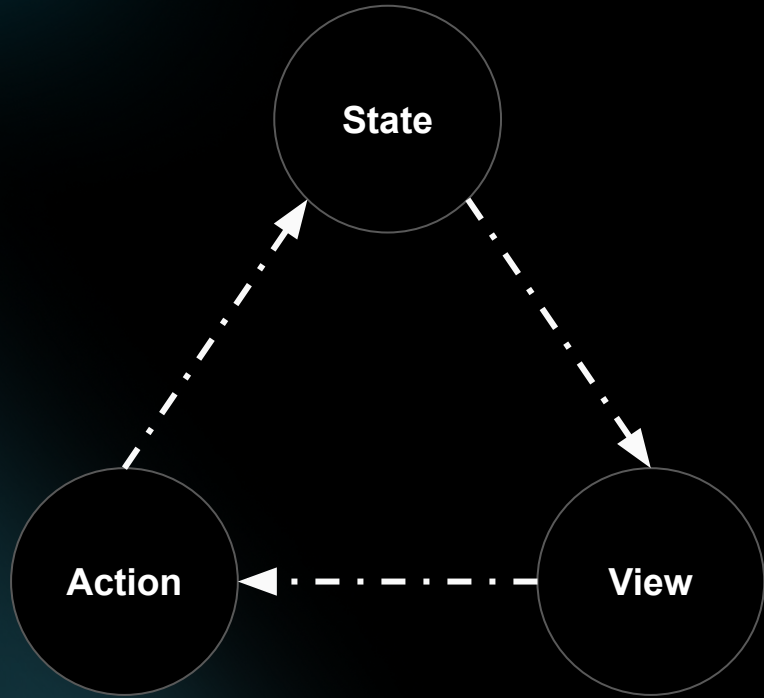
Unidirectional data flow



Unidirectional data flow



Bij state management
wijzigt de view dus nooit
rechtstreeks de data, maar
triggert die een verandering
in de state door een actie.



Waarom

**Goede state management zorgt voor
een duidelijke, voorspelbare en
overzichtelijke applicatie**

State is er altijd, state management maakt deze zichtbaar en behapbaar

Bijvoorbeeld, een API call heeft altijd een zekere loading time, door state management kunnen we die op een generieke en uniforme manier visualiseren door bijvoorbeeld loading spinners doorheen de hele applicatie.

Voordelen van state management

Voorspelbaarheid

Testbaarheid

Onderhoudbaarheid

Uitbreidbaarheid

Uniformiteit

Single source of truth

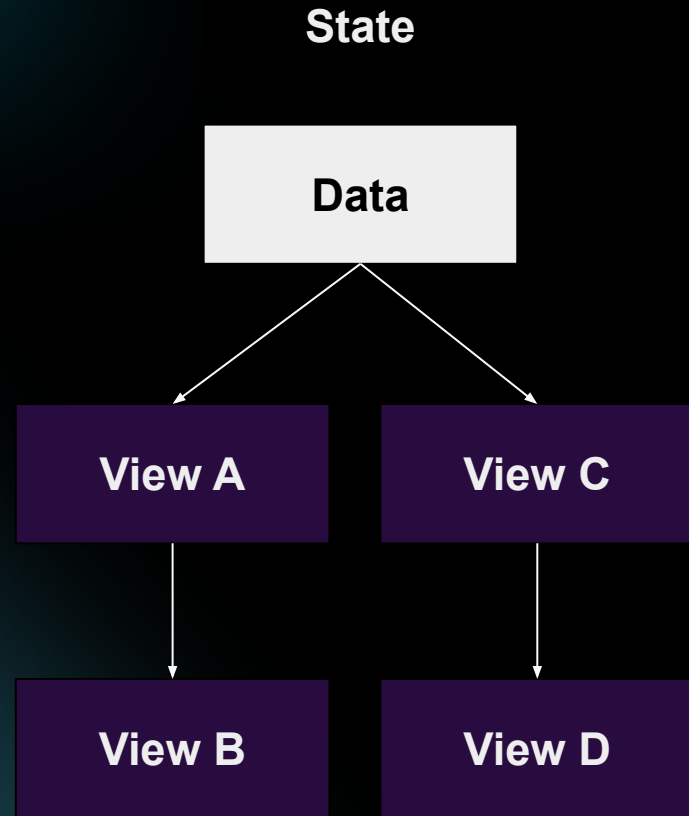
Testbaarheid

Kleine helpers (o.a. reducers & selectors)

Pure functions

Voorspelbaar

“Single source of truth”



Door state management
hebben we een goed inzicht
in de **data-flow**.

Waar de data vandaan komt

Hoe de data zich **verplaatst**
doorheen de views

Hoe de data zich **verhoudt**
tussen de views

**State management biedt
dus duidelijke voordelen.**

Hebben we een beter inzicht in
de **structuur van de applicatie**

Kunnen we **gemakkelijker
debuggen**

Kunnen we een **performantere
applicatie** opbouwen

**Bovendien kan de state een
extra cache laag vormen**

Bij intensieve API calls met weinig veranderende data, kan het bewaren van de data in de state helpen bij het reduceren van het aantal calls.

De state werkt dan als een soort
“backend van de frontend”.

Nadelen van state management

Extra boilerplate

(Leercurve)

Wanneer is state management **overbodig**?

Kleine / beperkte applicatie

Geen / weinig te verwerken data

Eigen systeem voor specifieke
toepassing

Hoe

**Er zijn verscheidene mogelijkheden,
dus bezint voor je begint!**

Naast een eigen
implementatie zijn er
verscheidene **open source**
mogelijkheden.

Redux
Akita
MobX
Recoil
NgRx...

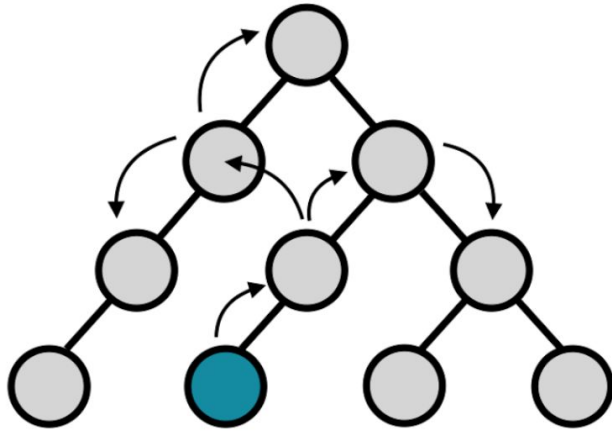
Traditioneel werd er het vaakst
gekozen voor een implementatie van
redux.

Redux is een **predictable state container** voor **Typescript apps**.

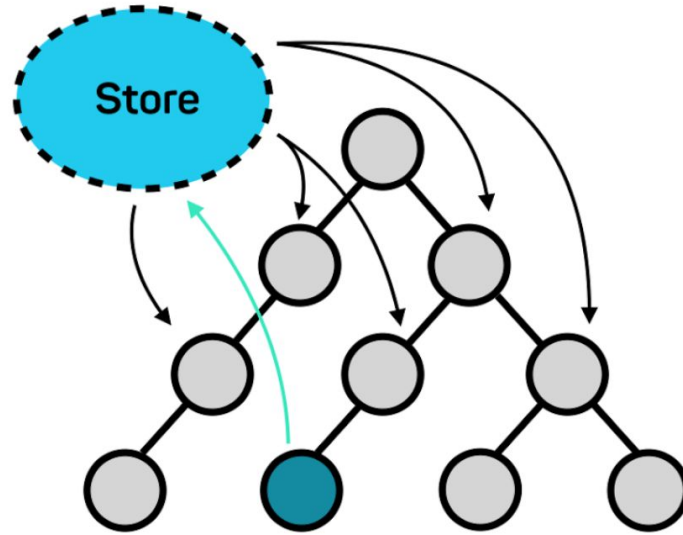
Redux is zorgt voor een voorspelbare state management en is volledig **framework agnostisch**.

Dit betekent dat Redux zowel in Angular, Vue, React, etc. kan toegepast worden.

Without Redux



With Redux



Component initiating change

Redux bestaat uit 6 basis onderdelen.

- Store
- State
- Selector
- Action
- Dispatch function
- Reducer

Het startpunt van Redux state management is de **store**.

De store houdt de data bij en coördineert wat er met de data moet gebeuren.

De store is een object met enkele **helper functions**. Aan de hand van de **state** houdt de store de data bij.

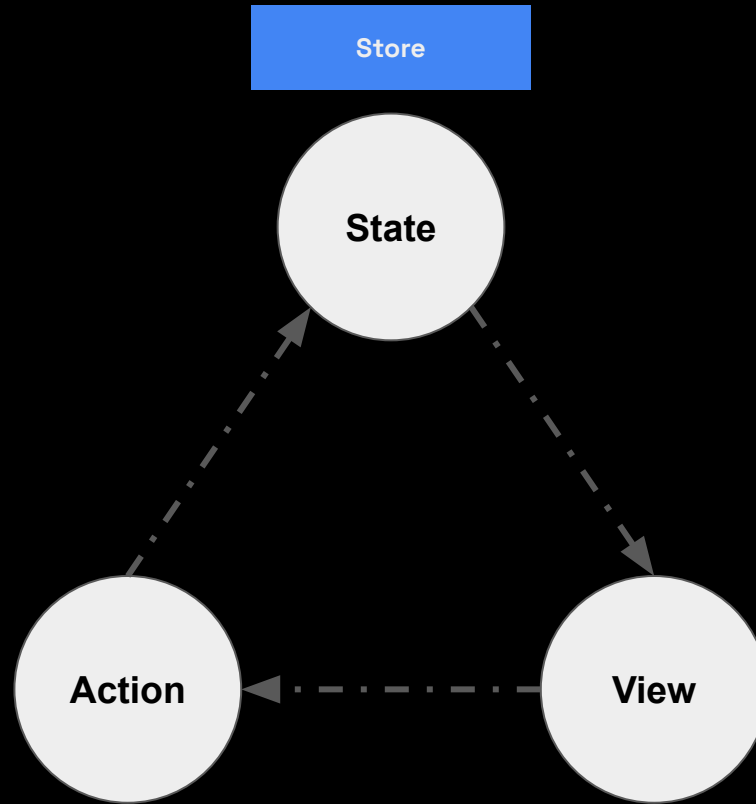

```
type Store = {  
  dispatch: Dispatch  
  getState: () => State  
  subscribe: (listener: () => void) => () => void  
  replaceReducer: (reducer: Reducer) => void  
}
```

State management draait natuurlijk
om **state**, wat door de store beheert.

**De state is een
representatie van de
status van een
applicatie.**

De state is een **key-value object** waarin we data opslaan die de status van de applicatie weergeeft.

De state wordt beheerd door de store, we interageren dus **nooit rechtstreeks** met de state.



```
type State = any
```

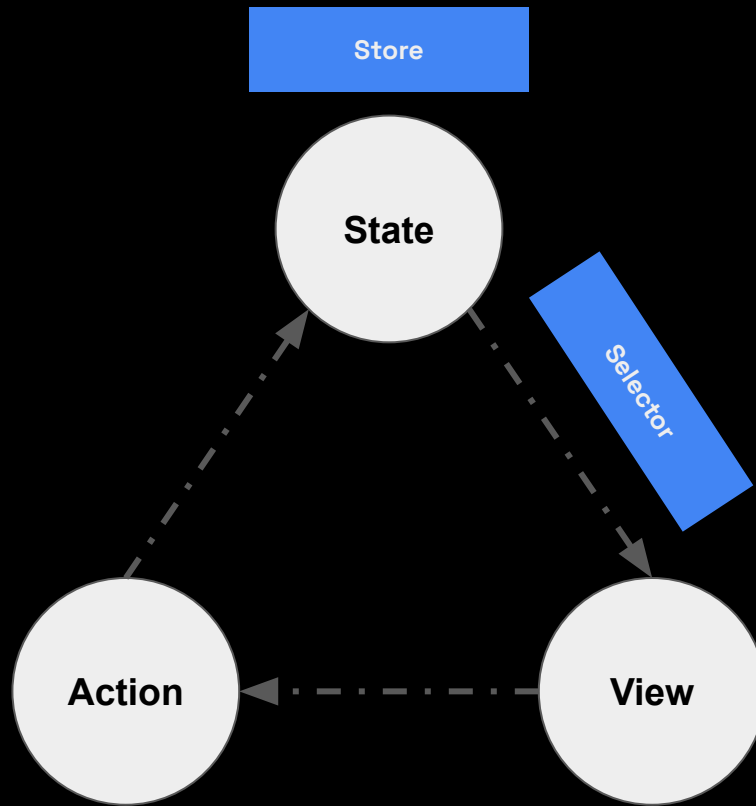
```
// Example:
```

```
const state = {  
  activeUser: { name: "Jasper" },  
  counter: 1,  
  darkMode: false,  
};
```

Om een stuk van de state op te
halen, maken gebruik van de
selector.

**Een selector is een
selectie function die
interageert met de
store.**

De selector kan zowel een de hele state als een stuk van de state ophalen van de store. Een stuk van de state noemen we een **slice**.




```
const allTodos = (state) => state.todos
```

**We interageren nooit rechtstreeks
met de state, we sturen de state aan
via de store aan de hand van
actions.**

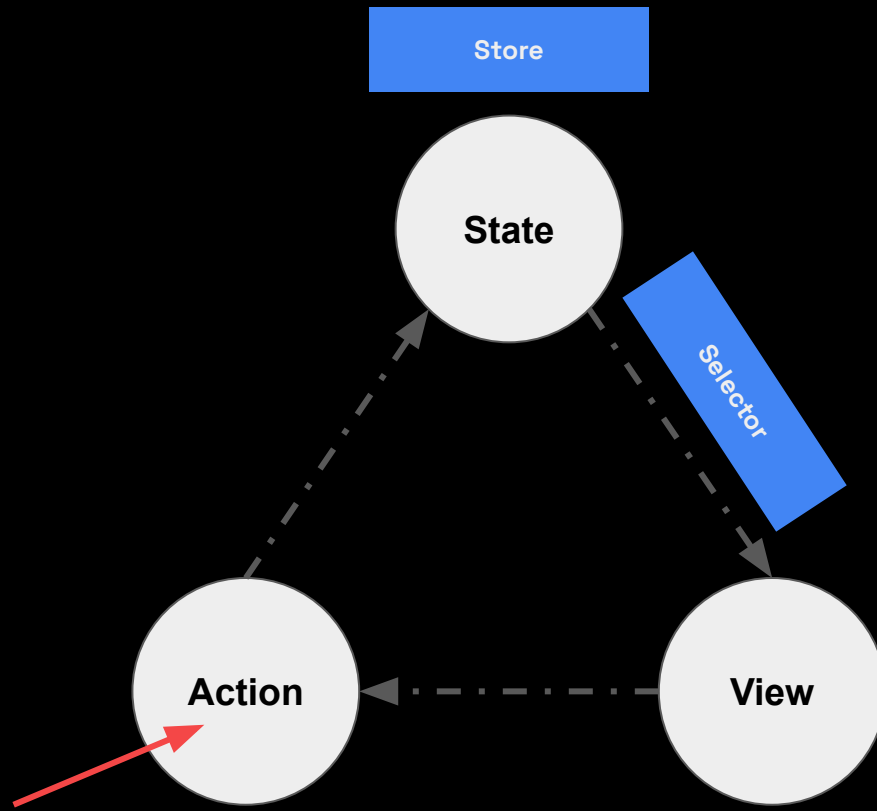
Een action
vergelijken we met
een **boodschap** die
we **aan de store**
geven.

Net als de store en de state, is een action ook opnieuw een object. Daar staat de **type** property centraal.

Een action past dus niet zelf de state aan, we zien het eerder als een “**bericht aan de store**” met een actie die de store moet uitvoeren.

**Aan een action
kunnen we ook
steeds een **payload**
meegeven.**

Een payload staat ons toe om **data mee te
geven aan de store**, die we dan later
kunnen gebruiken.

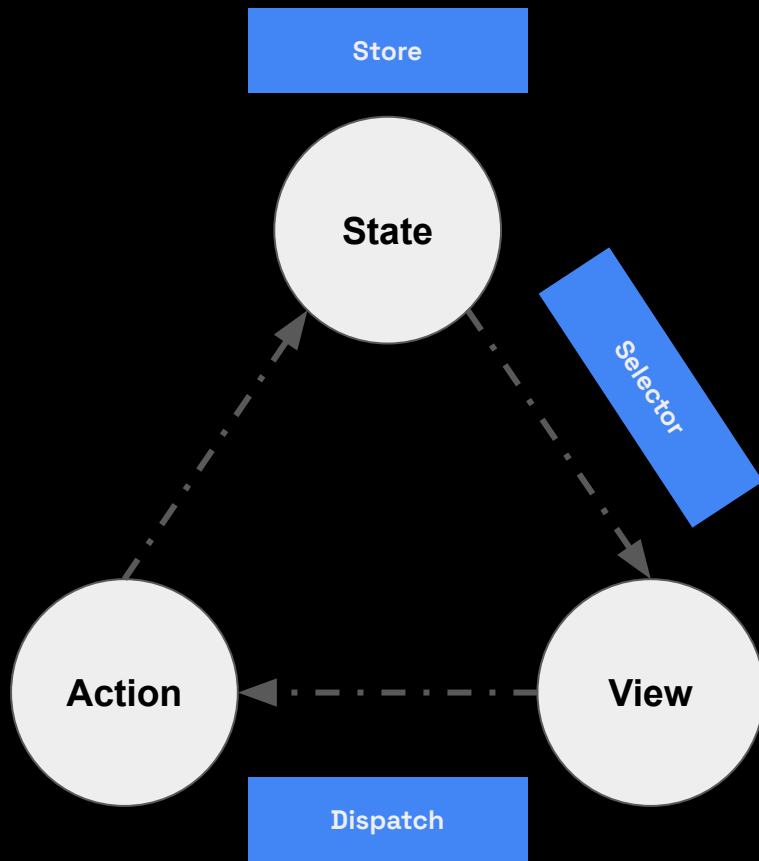


```
const myAction = {  
  type: "MyAction",  
  payload: {  
    user: { name: "Jasper" },  
  },  
};
```

Als we actions zien als berichten,
moeten we die ook versturen naar
de store. Dit noemen we **dispatchen**.

**Dispatch is een
synchrone functie
die een action
doorstuurt naar de
store.**

We zien het dus als de verzender van de action, die de intentie van de gebruiker/component overbrengt naar de store.



```
type BaseDispatch = (a: Action) => Action  
type Dispatch = (a: Action | AsyncAction) => any
```

**Voordien dispatchten we actions
naar de store. Deze opdrachten
worden dan verwerkt in **reducers**.**

Reducers zijn pure functions.

Een pure function is een function die data binnen krijgt en vervolgens steeds data returned.

Een reducer luistert
naar de **action** en **de**
huidige state.

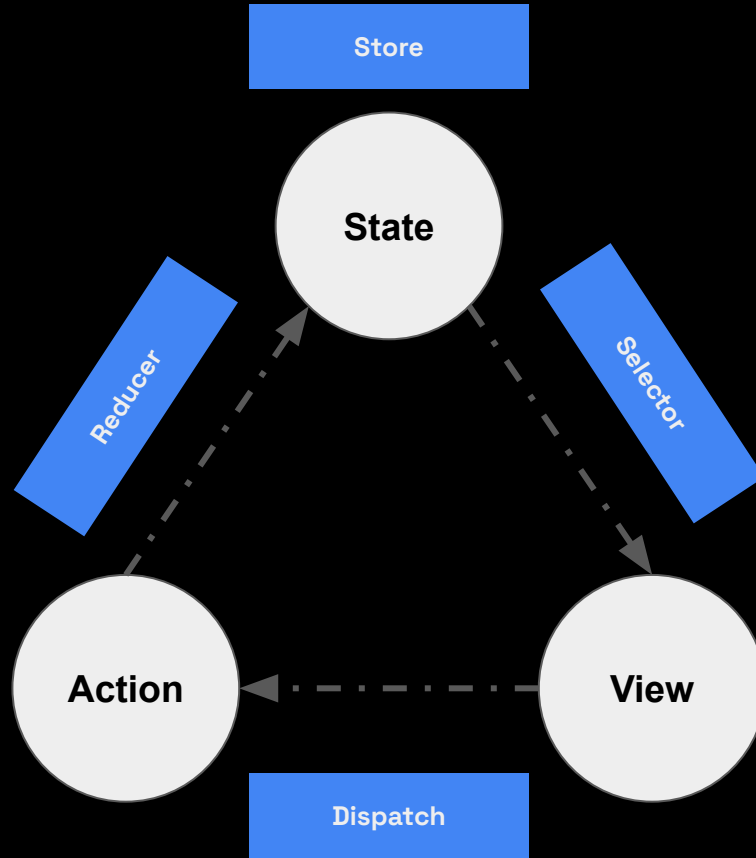
Voor iedere **action (boodschap)** die we
schrijven, schrijven we ook een **reducer**
(ontvanger).

Omdat de reducer iets moet uitvoeren op
de state, krijgt die naast de action ook **de**
huidige state mee.

Reducers **returnen** **altijd state.**

Een reducer zal aan de hand van een action de state teruggeven. Deze state kan ofwel de **huidige state** zijn, of een **geupdate versie** van de state.

In de meeste gevallen zal het gaan om een geupdate versie van de state.



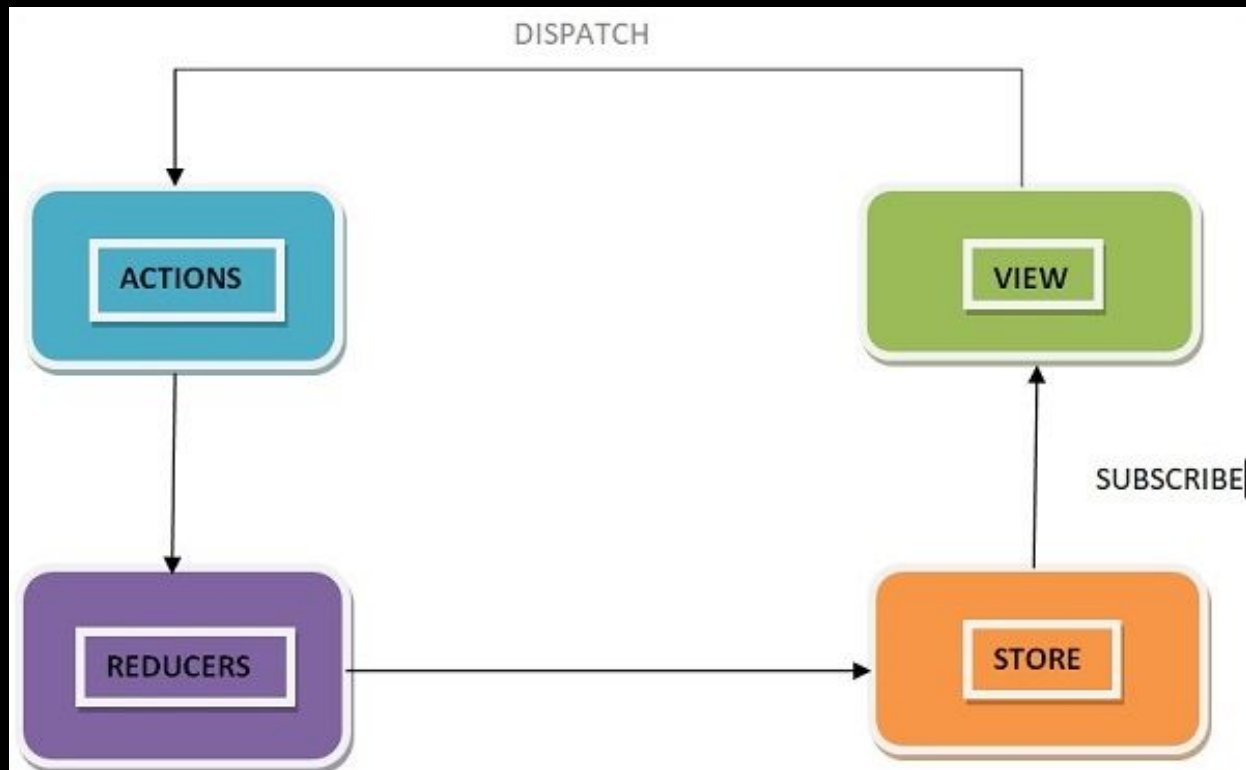
```
type Reducer<S, A> = (state: S, action: A) => S

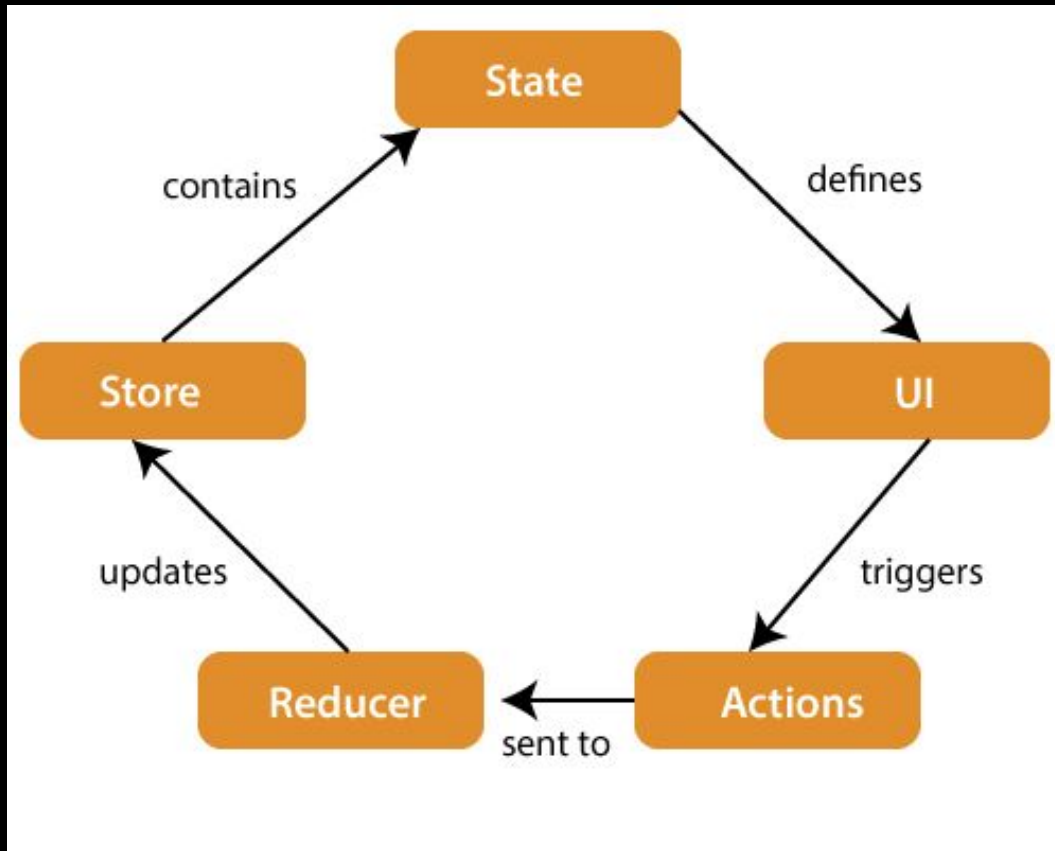
const myReducer = (state, action) => {

  // Return new state for specific action
  if (action.type === "myAction") {
    return {
      ...state,
      user: action.payload.user,
    }
  }

  // Always return state!
  return state;
}
```


We kennen nu alle onderdelen van
Redux. We bekijken in het overzicht
hoe die met elkaar interageren.





De voor- en nadelen van Redux

Redux voorziet basis implementaties om zowel immutability als unidirectional data flow te garanderen

Redux komt met een devtool **Redux Devtools** waarmee we alle acties op de state kunnen tracken

Door het gebruik van reducers, actions en selectors kan Redux wel heel wat boilerplate veroorzaken

ACTIONSREPORTSSETTINGS

RESETREVERTSWEEPCOMMIT

Autoselect instances

filter...

@@INIT11:30:52.35

Navigation/NAVIGATE+00:16.27

Navigation/BACK+00:00.84

Diff

ActionStateDiffTest

TreeRaw

▼ nav (pin)

index (pin): + => 0

► routes (pin): { 1: {...} }

INSPECTORLOG MONITORCHART

>_

